

Pluribus, in this codebase

from the idea to the code — a guided tour of `trainer/pluribus`

A tabular blueprint (**no neural network**) plus depth-limited search, implemented for heads-up no-limit hold'em

Companion to `docs/modern_poker_ai_tutorial.pdf` — that document is the *theory* (its Pluribus section); this one is the *map from theory to code*.

Intuition

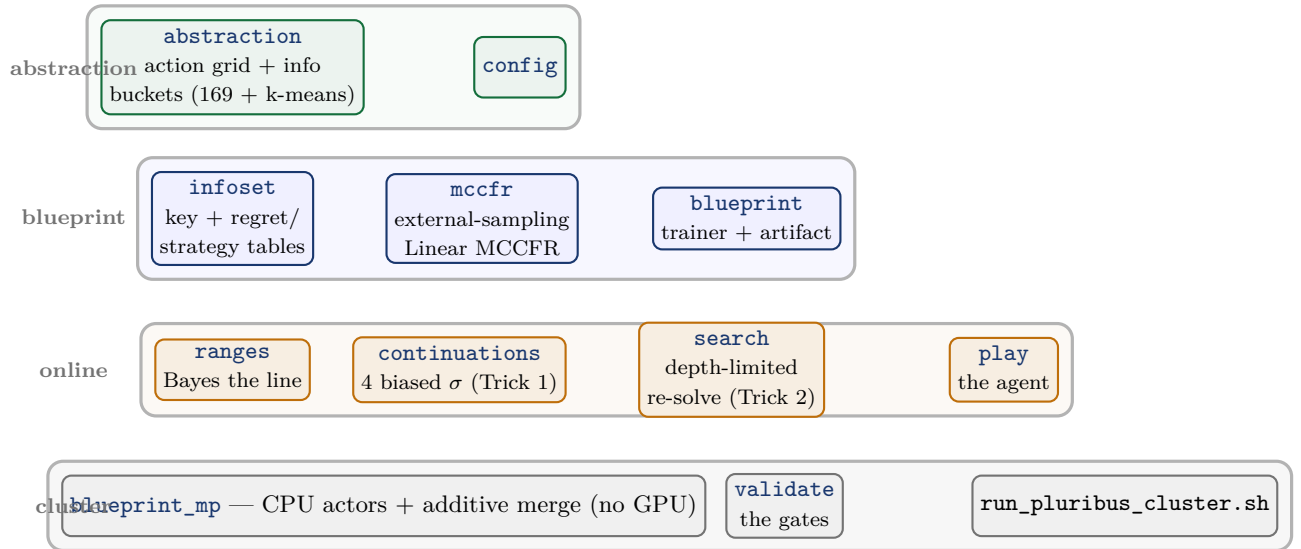
Pluribus's one idea (tutorial, §Pluribus): a coarse offline **blueprint** strategy — computed by **Linear MCCFR** over an *abstracted* game and stored in **regret tables**, with *no neural network* — plus cheap *depth-limited* online **re-solving** is enough to play superhuman poker. It famously trained on a **64-core CPU server, no GPU**: the scarce resource is RAM for the tables, not FLOPs. This codebase implements that *method* over the project's heads-up engine (the engine is 2-player; ReBeL was ported the same way). Every Pluribus concept below is named with the file and function that realizes it.

Contents

1	High-level view: the module map	1
2	Information & action abstraction (<code>abstraction.py</code>)	2
3	The blueprint: tabular Linear MCCFR (<code>infoset.py</code> , <code>mccfr.py</code> , <code>blueprint.py</code>)	3
4	Online re-solving (<code>ranges.py</code> , <code>continuations.py</code> , <code>search.py</code> , <code>play.py</code>)	4
5	The cluster trainer (<code>blueprint_mp.py</code>)	4
6	Correspondence at a glance, and what is validated	5

1 High-level view: the module map

`trainer/pluribus/` is pure Python. It reuses the pybind engine (`pokertrainer_engine`) to enumerate legal bets and evaluate hands, and reuses `trainer/rebel_py` for the *search* primitives (the 1326-combo hand index, showdown values, the vectorized CFR-D subgame solver and the best-response gate) — ReBeL and Pluribus share the search; what differs is that Pluribus's blueprint is **tables, not a net**, and its leaves are valued by **continuation strategies**, not a value network.



The phases (how it was built and validated).

Phase	What	Gate (proof it is correct)
1	tabular Linear-MCCFR blueprint + abstraction (no search)	10bb push/fold = Nash oracle (SB jam 58.0 vs 58.1%, BB call 37.6 vs 37.3%)
2	depth-limited online re-solving (Tricks 1 & 2 + nested)	river re-solve → 0.002 bb/hand exploit. (whole 1326-hand range)
3	cluster blueprint trainer (CPU actors, no GPU)	64-core actor pool; additive table merge; checkpoints round-trip

Intuition

What is Pluribus's, and what is shared with ReBeL? The blueprint half (`abstraction`, `infoaset`, `mccfr`, `blueprint`) is pure Pluribus and has no ReBeL analogue — it is the “no neural network” branch. The search half reuses `rebel_py`'s vectorized CFR-D solver and exploitability gate (the math of solving a public subgame over the 1326-combo range is identical); Pluribus only swaps the value-net leaf for a continuation-strategy leaf (Trick 1) and feeds it Bayes ranges built from the *tabular* blueprint.

2 Information & action abstraction (`abstraction.py`)

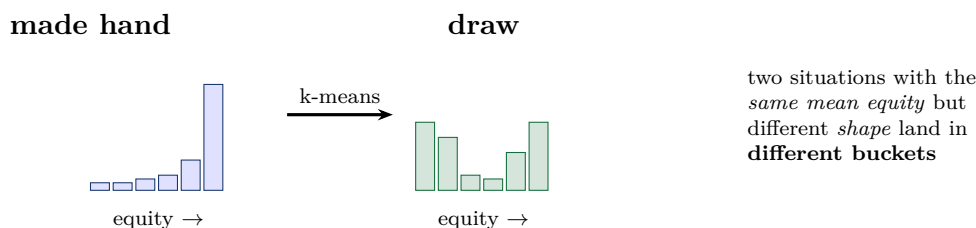
Two abstractions shrink 10^{161} situations to a table that fits in RAM.

Action abstraction. `ActionAbstraction.actions(state, legal)` restricts the engine's 11 actions to a small grid. The "pushfold" preset is the Phase-1 gate (preflop jam/fold for the opener, call/fold facing an all-in — exactly the game the Nash oracle solves); "discrete" is a per-street pot-fraction grid that always force-includes fold/call/all-in so the tree closes. Betting is hole-independent, so this is a function of the public state only.

Information abstraction. `InfoAbstraction.bucket_cards(street, board, c0, c1)` maps a situation to a bucket id.

- **Preflop is lossless:** the 169 strategic classes (`evaluate.preflop_equity.class_of`), shared with the push/fold oracle.

- **Postflop is k-means on equity-distribution histograms.** A hand on a board is described by the *shape* of its equity across runouts — so a made hand (mass near 1) and a draw (bimodal) separate even at equal raw equity. Centroids are learned on a board *sample* (`learn_street_buckets`); every situation is assigned to its nearest centroid at run time, cached.



Watch out

The Phase-1 push/fold gate is *preflop-only*, so it never touches the k-means path. A full “discrete” postflop game needs a learned bucket cache; without one, `bucket_cards` falls back to the (coarse) preflop class and prints a warning. Building the real per-street buckets is the offline precompute that the cluster run owns.

3 The blueprint: tabular Linear MCCFR (`infoset.py`, `mccfr.py`, `blueprint.py`)

Abstract infoset key (`infoset_key`): (`street`, `to_act`, `bucket`, `history_token`), where the history token is the chronological tuple of engine `ActionType` ints applied so far. Two situations mapping to the same key share a strategy — that sharing is what makes the table small. Regret and strategy-sum are plain dicts `key → ndarray[11]` (`Tables`), **sparse** and **additive** (the property the cluster merge needs).

`mccfr.traverse` is one external-sampling Linear-MCCFR traversal — the same scheme as the repo’s Deep CFR (`cfr_coro`), but writing to tables instead of neural-net buffers:

Listing 1: `mccfr.py` — the traversal. σ is read by regret matching on the stored regrets; it is accumulated for the average strategy **ONLY** at opponent nodes (Brown 2019, Alg. 1), where sampling visits $\propto$ the opponent’s own reach.

```
def traverse(env, traverser, weight, tables, abstraction, rng, allin_samples=0):
    if env.is_terminal():
        # variance reduction: value an all-in showdown by Monte-Carlo equity
        # over K runouts, not the single board the engine happened to deal.
        if allin_samples and both_all_in(env): return allin_equity(env, ...)
        return env.payoffs_bb()[traverser]
    key = infoset_key(env.state(), abstraction.info)
    sigma = regret_match(tables.regret[key], abstract_legal_mask)
    if actor != traverser:
        # OPPONENT node
        tables.strat[key] += weight * sigma # accumulate avg strategy (Linear t)
        a = sample(sigma); env.step_action(a) # external sampling
        return traverse(env, ...)
    v = zeros(11) # TRAVERSER node: branch every action
    for a in abstract_legal:
        v[a] = traverse(env.clone().step(a), ...)
    v_state = (sigma * v).sum()
    tables.regret[key] += weight * (v - v_state) # Linear-CFR regret (weight = t)
    return v_state
```

Intuition

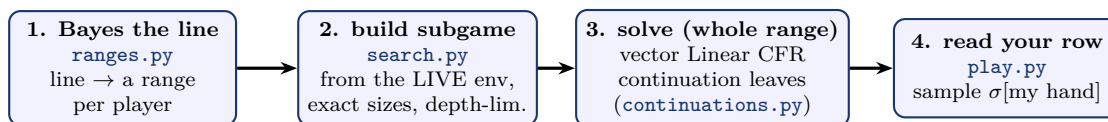
Why the Monte-Carlo all-in equity matters. A called all-in is settled by the board runout, which swings the payoff by $\pm$ the whole stack. Taking the single board the engine dealt makes the regret of a *weakly*-dominant jam (e.g. a hand that is $+0.3$ bb to shove) impossible to pin down at a feasible iteration count. Averaging the all-in over K sampled runouts (`_allin_equity_payoff`) is a standard, oracle-independent variance reduction: it converges to the same equilibrium but makes the push/fold gate *fast and tight*. With it, the 10bb blueprint matches the Nash oracle’s aggregate ranges to $\sim 0.1\text{--}0.3\%$ (without it, the same iteration budget leaves $\sim 10\%$ of weakly-decisive hands mis-resolved).

`train_blueprint` runs T iterations, each a traverser==SB pass and a traverser==BB pass (Linear-CFR weight = t), so both players’ regret *and* strategy tables fill. `Blueprint` wraps the result: it computes the normalized average strategy on demand, masked to the abstract-legal actions of the queried state (unseen infosets fall back to uniform).

10bb push/fold (300k iters, $K=32$)	blueprint	Nash oracle
SB jam (combo-weighted)	58.0%	58.1%
BB call (combo-weighted)	37.6%	37.3%
decisive-hand misplay (SB / BB)	2.7% / 0.9%	—

4 Online re-solving (`ranges.py`, `continuations.py`, `search.py`, `play.py`)

The blueprint alone is too coarse to be strong; at the table the agent re-solves the actual spot. The pipeline (`search.resolve`) is the tutorial’s four boxes:



Bayes ranges (`ranges.py`). `bayes_ranges` replays the public history under the blueprint: `range_p[h]` $\propto \prod \sigma(\text{action} \mid \text{infoset of } p \text{ holding } h)$, card-removal masked. Hands that fit the line get boosted; hands the blueprint would (almost) never play this way go to ~ 0 . (Validated: after an SB jam, the SB range puts $\sim 10^3\times$ more mass on AA than on 72o.)

Trick 2 — solve for the whole range (`search.py`). `build_engine_subgame` enumerates the abstract betting tree *from the agent’s live engine state* (so pot/stacks/exact sizes — including an opponent’s off-tree size, already in the state — are exact, no rounding) into a `rebel.py` Subgame, which `CfrSolver` solves over all 1326 hands at once. The agent reads off the row of its actual hand only at the end (`_root_action_probs`). On the river this is exact (true terminals, no continuations): re-solving reaches **0.002 bb/hand** exploitability.

Trick 1 — continuation strategies (`continuations.py`). Before the river, a leaf (the street boundary) is not the end of the hand, so its value depends on how play continues. Valuing it with a single fixed continuation lets the search above cheat. `bias_strategy` builds the 4 continuations (blueprint, fold/call/raise-biased — multiply one action family’s probability by a constant and renormalize, support preserved); the opponent’s choice among them is folded into the CFR solve at the leaf, so the equilibrium is robust to all 4^2 continuations.

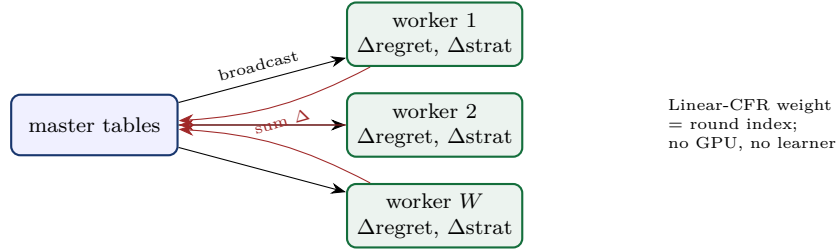
Nested re-solving. An opponent bet outside the agent’s size grid is already in the live engine state (applied exactly, via `step_raise_to`), so re-solving *from that state* prices the exact size with no rounding — just call `resolve` again.

Listing 2: `play.py` — the agent. Preflop (dense abstraction) plays the blueprint directly; on the river it re-solves the whole range and reads its row. Postflop search needs a postflop-trained blueprint to value continuation leaves.

```
def act(self, env):
    st = env.state()
    if int(st.street) not in self.search_streets:      # preflop / no postflop net
        return self.bp.sample(st, self.rng)          # blueprint, look at my cards
    probs, _, _ = resolve(env, self.bp, self.cfg)      # solve the WHOLE range
    h = hand_index(*my_hole_cards)
    return sample(probs[h])                          # my row, only at the end
```

5 The cluster trainer (`blueprint_mp.py`)

Tabular MCCFR is CPU/RAM-bound — the 64-core, no-GPU profile Pluribus trained on. `train_blueprint_mp` is synchronous data-parallel CFR: each round ships the master tables to W workers; each worker deep-copies them, runs a batch of traversals (adapting locally), and returns only its **increments**; the master sums them. Because regret/strategy-sum are additive, summing W workers’ increments $\approx$ doing $W \times$ the traversals of sequential CFR.



`scripts/run_pluribus_cluster.sh` launches it inside the Singularity image with *no -nv* (the blueprint never touches CUDA). At 700 GB-RAM / 64-core scale the keyspace would be *sharded* across workers (each owns a hash bucket, no per-round broadcast) — the additive merge here is the primitive that extension builds on.

6 Correspondence at a glance, and what is validated

Pluribus idea (tutorial)	file · function	status
information abstraction (k-means equity)	<code>abstraction.bucket_cards</code>	impl.; postflop k-means not yet exercised
action abstraction (few bet sizes)	<code>abstraction.ActionAbstraction</code>	validated (pushfold gate)
Linear MCCFR, external sampling	<code>mccfr.traverse</code>	validated vs Nash oracle
blueprint = avg strategy (tables)	<code>blueprint.Blueprint</code>	validated (push/fold)
Bayes the line $\rightarrow$ ranges	<code>ranges.bayes_ranges</code>	validated (post-jam range)
Trick 1: continuation strategies	<code>continuations.bias_strategy</code>	impl. + unit-tested
Trick 2: solve the whole range	<code>search.resolve</code>	validated (river $\rightarrow$ 0)
nested re-solving (off-tree sizes)	<code>search</code> (live-state re-solve)	impl.
the play-time loop	<code>play.PluribusAgent</code>	impl. (river + preflop)
64-core blueprint, no GPU	<code>blueprint_mp</code>	validated (parallel merge)

Hard gates (run them).

```
PYTHONPATH=engine/build:trainer python -m pluribus.validate # both phases
PYTHONPATH=engine/build:trainer python trainer/tests/test_pluribus_phase1.py
PYTHONPATH=engine/build:trainer python trainer/tests/test_pluribus_phase2.py
```

Phase 1 (`validate_pushfold`) trains a 10bb push/fold blueprint and checks it reproduces `evaluate.pushfold_score` Nash equilibrium (aggregate ranges within $\sim 0.3\%$, decisive-hand misplay $< 3\%$). Phase 2 (`validate_river_resolve`) re-solves several river spots over the whole 1326-hand range and asserts < 0.05 bb/hand exploitability (observed ~ 0.002). The mixed-band frequency differences from the exact oracle are expected sampling variance, not error (Pluribus’s blueprint is itself a sampled approximation; the real run uses far more iterations).

Out of scope (first pass). A true 6-max engine (the engine is heads-up; Pluribus’s *method* is what is ported); suit-isomorphism beyond k-means; a C++ port for max throughput; cross-street continuation rollouts for flop/turn search (the river re-solve and the continuation *gadget* are implemented; wiring a postflop-trained blueprint through the leaf rollout is the documented next step); and a UI/Slumbot seat for the agent (the `play` policy is designed to drop in, as the ReBeL net was).

An engine bug fixed along the way. External-sampling MCCFR calls `Env.clone()` once per branched action. `clone()` was constructing a throwaway `Env` via the public constructor, which rebuilt the entire `HandEvaluator` lookup table (≈ 2 ms) only to discard it — a silent tax on *all* CFR traversal in the repo (Deep CFR clones too). A one-line private tag constructor that builds nothing dropped `clone()` from $1950\ \mu\text{s}$ to $1.07\ \mu\text{s}$ ($\sim 1800\times$), turning the blueprint from 150 to 4500 iters/s.

Paper. Brown & Sandholm, *Superhuman AI for multiplayer poker*, Science 365 (2019). **Code.** `trainer/pluribus/` (+ `trainer/rebel_py` for the shared search primitives).